

Middle Layers Know Where to Click: Coordinate Serialization Bottlenecks in GUI Agents

Anonymous submission

Abstract

Autoregressive GUI agents ground instructions by generating screen coordinates, conflating target localization with coordinate serialization. We show that these computations peak at different depths: spatial grounding is best decoded from intermediate transformer layers and often degrades near the output, while coordinate-token predictability continues improving in later layers. We call this mismatch the **coordinate serialization bottleneck**. Across four GUI agents and two backbone families, a lightweight per-layer spatial probe and a parameter-free serialization lens reveal the same spatial–serialization dissociation. An instruction-switch counterfactual rules out generic visual saliency, and the bottleneck is strongest on high-resolution layouts with small targets. Motivated by these findings, we introduce ZWERGE-UI, a frozen-backbone retrofit that reads intermediate-layer spatial posteriors through lightweight cross-attention probes ($\approx 42\text{M}$ parameters, $\sim 0.5\%$ of an 8B backbone) and a small cross-layer fusion head ($< 200\text{K}$ parameters), requiring no backbone modification or autoregressive coordinate decoding. The largest gains occur when the intermediate–final spatial gap is largest, linking improvements to bottleneck severity.

Introduction

Visual GUI agents are typically trained to ground natural-language instructions by generating screen coordinates or action protocol tokens (????). This end-to-end formulation conflates two computations: *locating* the target widget in screen space and *serializing* that location into a coordinate or action vocabulary. Since the final action is decoded from the last representation, GUI grounding is often analyzed only through the final generated coordinates, leaving unclear whether spatial localization and coordinate serialization emerge at the same depth inside the model.

Prior work has exploited attention maps and prefill-stage signals to aid localization (???), while analyses of vision transformers show that final-layer representations may lose spatial fidelity (???). However, these lines of work do not ask where *within* the prefill computation spatial grounding is most accessible in autoregressive GUI agents. At the same

time, GUI-agent training studies reveal a tension between chain-of-thought reasoning and precise localization (????), motivating perception–reasoning decoupling (?). We operationalize this perspective by freezing the backbone and reading spatial beliefs directly from intermediate activations.

We study this question by probing intermediate layers of representative GUI agents with lightweight frozen-backbone readouts. Our analysis reveals a consistent non-monotonic trajectory: spatial grounding improves through early and middle layers, peaks at intermediate depths, and then often degrades toward the final layer. In contrast, a complementary *serialization lens*—teacher-forced coordinate-token likelihood without additional parameters—shows that coordinate tokens become most predictable in later layers. This mismatch suggests that GUI agents first form an internal spatial intent and only later translate it into output tokens. We call this phenomenon the **coordinate serialization bottleneck**.

This bottleneck is not a generic saliency effect. In an instruction-switch counterfactual, changing only the instruction on the same screen shifts the mid-layer spatial posterior from one target to another, indicating that the posterior is instruction-conditioned. The effect is also most pronounced in high-resolution layouts with small targets, where precise screen-space localization is most critical. Together, these findings suggest that the final representation is not always the best place to decode GUI grounding: late layers increasingly specialize in expressing coordinates, while intermediate layers preserve more spatially discriminative information.

Motivated by this observation, we introduce ZWERGE-UI, a frozen-backbone retrofit for existing GUI agents. ZWERGE-UI attaches lightweight cross-attention probes to intermediate layers to recover layerwise spatial posteriors, and optionally combines them with a small cross-layer fusion head. The probes serve a dual role: they provide the analytical lens used to reveal the bottleneck, and they form the grounding component used at inference. Importantly, ZWERGE-UI does not modify the agent backbone or its reasoning pathway.

We evaluate ZWERGE-UI on four GUI agents spanning two backbone families, Qwen2.5-VL (?) and Qwen3-VL (?), across five GUI grounding benchmarks: ScreenSpot-Pro (?), ScreenSpot-v2 (?), OSWorld-G (?), MMBench-GUI (?), and UI-Vision (?). Across models and datasets, the strongest spatial readout consistently appears before the final

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

layer. Moreover, models with sharper final-layer degradation benefit more from the intermediate-layer retrofit, linking the empirical gains of ZWERGE-UI to the severity of the coordinate serialization bottleneck.

In summary, our contributions are as follows:

- We identify the **coordinate serialization bottleneck** in GUI agents: intermediate layers encode more accurate instruction-conditioned spatial posteriors than final layers, while later layers better predict coordinate/action tokens.
- We introduce two complementary diagnostic lenses—a frozen-backbone *spatial lens* and a parameter-free *serialization lens*—and use them to show a robust depth mismatch between spatial grounding and coordinate serialization across four GUI agents.
- We present ZWERGE-UI, a lightweight frozen-backbone retrofit that recovers intermediate-layer spatial signals through cross-attention probes and improves GUI grounding across five benchmarks without modifying the underlying agent backbone.

The Coordinate Serialization Bottleneck

We investigate *where* GUI grounding signals are encoded across transformer layers, building toward four findings that motivate our method.

Experimental setup. We study four GUI agents spanning two backbone families (Qwen2.5-VL (?) and Qwen3-VL (?) on ScreenSpot-Pro (?) and ScreenSpot-v2 (?). Spatial grounding quality is measured via a lightweight per-layer grounding head (Stage 1 of §??; $\sim 0.5\%$ of backbone parameters) trained with the backbone completely frozen, so it introduces no new spatial knowledge. Coordinate serialization quality is measured via the backbone’s native language model head under teacher forcing, with no additional parameters.

Spatial Grounding Peaks at Intermediate Layers

Autoregressive GUI agents implicitly assume final-layer representations carry the richest spatial information. Figure ?? (blue curves) challenges this: the spatial signal peaks at intermediate depths, then declines toward the output. This non-monotonic trajectory is consistent across both backbone families, ruling out any progressive-refinement interpretation.

Finding 1. *Intermediate transformer layers are consistently better spatial decoders than the final layer.*

A Dissociation Between Spatial Localization and Coordinate Serialization

If late layers are not spatial refiners, what do they do? We hypothesize they are dedicated to *coordinate serialization*, translating a latent spatial intent into the vocabulary-level token sequence required by the autoregressive head. We test this with a *serialization lens*: coordinate token log-likelihood measured via teacher-forced inference at each layer.

Figure ?? (red curves) reveals a striking dissociation. Both signals rise together through early layers, then diverge: spatial quality peaks and falls, while coordinate log-likelihood continues climbing to a later plateau. The lag between the spatial peak and the serialization plateau is consistent across all four models and both backbone families, providing direct evidence for a **coordinate serialization bottleneck**.

Finding 2. *The layer that best decodes the target region consistently precedes the layer that best predicts the coordinate token sequence. Late layers are coordinate serializers, not spatial refiners.*

The Mid-Layer Posterior Is Instruction-Conditioned, Not Visual Saliency

A skeptical reading is that mid-layer spatial signals merely reflect *visual saliency*: prominent targets appear highlighted regardless of the instruction. We design a counterfactual to falsify this: for pairs of records sharing the same screen but targeting disjoint elements, we swap the instruction while keeping the image fixed and measure how much the spatial posterior shifts.

Figure ?? shows the posterior shifts dramatically. Switching the instruction collapses the spatial mass on the original target by a substantial margin, concentrated at exactly the intermediate layers where grounding quality peaks. This decisively rules out the saliency account: the mid-layer representation is an instruction-conditioned spatial posterior, not a passive saliency map.

Finding 3. *Swapping the instruction while keeping the image fixed causes a large shift in the mid-layer spatial posterior. The spatial signal is instruction-conditioned grounding, not visual saliency.*

The Bottleneck Is Amplified on High-Resolution Small-Target Layouts

We ask whether the bottleneck is *amplified* in the setting where GUI grounding is hardest: high-resolution screen-shots with small, densely packed targets.

We quantify severity via the *collapse gap*: peak-layer hit@1 minus final-layer hit@1. Table ?? shows the gap is consistently larger on ScreenSpot-Pro (high-resolution, small targets) than on ScreenSpot-v2 (mid-resolution, larger targets) across every model. This is not a difficulty artifact: even though absolute performance is higher on SS-v2, the relative degradation from peak to final layer is smaller. The serialization overhead is most costly precisely where spatial precision matters most.

Finding 4. *The collapse gap from peak to final layer is consistently larger on high-resolution benchmarks with small targets. The coordinate serialization bottleneck is most damaging precisely where spatial precision matters most.*

Taken together, the four findings motivate reading the spatial posterior from intermediate layers and using cross-layer

Spatial Grounding Peaks Before Coordinate Serialization

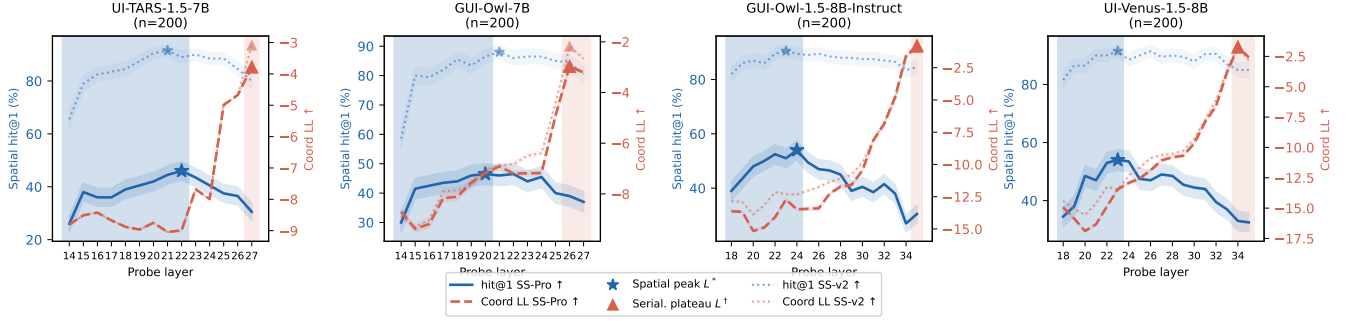


Figure 1: **Spatial localization and coordinate serialization peak at different depths.** Blue curves (left axis): spatial hit@1 $\uparrow$. Orange curves (right axis): coordinate token log-likelihood $\uparrow$. Both signals rise together, then diverge at the spatial peak L^* (blue star): spatial quality declines while serialization continues rising to $L^\dagger$ (orange triangle). The consistent lag $L^\dagger > L^*$ across all models constitutes direct evidence for the coordinate serialization bottleneck.

Instruction Switch Suppresses Old Target at Mid-Layers

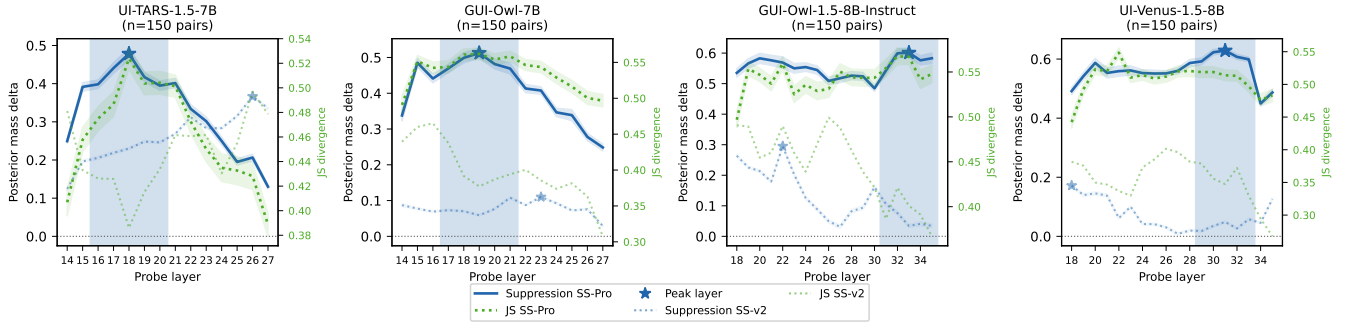


Figure 2: **Instruction-switch counterfactual: the mid-layer posterior is instruction-conditioned.** Blue curves: old-target suppression $\uparrow$. Orange curves: posterior JS divergence $\uparrow$. Both effects peak at the same intermediate layers as spatial grounding quality (cf. Fig. ??) and are substantially nonzero, ruling out visual saliency.

Model	SS-Pro			SS-v2		
	Pk	Fin	Gap $\downarrow$	Pk	Fin	Gap $\downarrow$
UI-TARS-1.5-7B	46.0	30.5	15.5	91.5	80.0	11.5
GUI-Owl-7B	46.5	37.0	9.5	88.0	80.5	7.5
GUI-Owl-1.5-8B	54.0	30.5	23.5	90.5	84.5	6.0
UI-Venus-1.5-8B	54.0	32.5	21.5	91.5	85.0	6.5

Table 1: **Collapse gap is larger on high-resolution layouts.** Pk = peak-layer hit@1; Fin = final-layer hit@1 (pp). SS-Pro gaps consistently exceed SS-v2 gaps.

fusion to select the optimal depth per sample, as described in §??.

ZWERGE-UI

The analysis in §?? reveals that the richest spatial representations reside at intermediate transformer layers, where the instruction-conditioned spatial posterior peaks before being overwritten by coordinate serialization. ZWERGE-UI

retrofits existing GUI agents to read this intermediate-layer signal without modifying the backbone. The Stage 1 per-layer probes trained here also serve as the readout heads used in all probe experiments in §??. Figure ?? provides an overview of this retrofit pipeline, connecting the prefill-time layerwise readout, per-layer spatial posteriors, and cross-layer fusion used in the following subsections.

Design Rationale

A dedicated readout head. The raw hidden states at intermediate layers *contain* spatial information (Finding 1) but are not calibrated for direct readout. A full-rank cross-attention probe trained on grounding examples with the backbone frozen is sufficient to decode this latent signal; it introduces no new spatial knowledge into the backbone.

Prefill-forced inference. Coordinate generation serializes location into up to eight individual number tokens, but the spatial intent is already encoded before decoding begins (Finding 2). Because the backbone is frozen, we prefill a

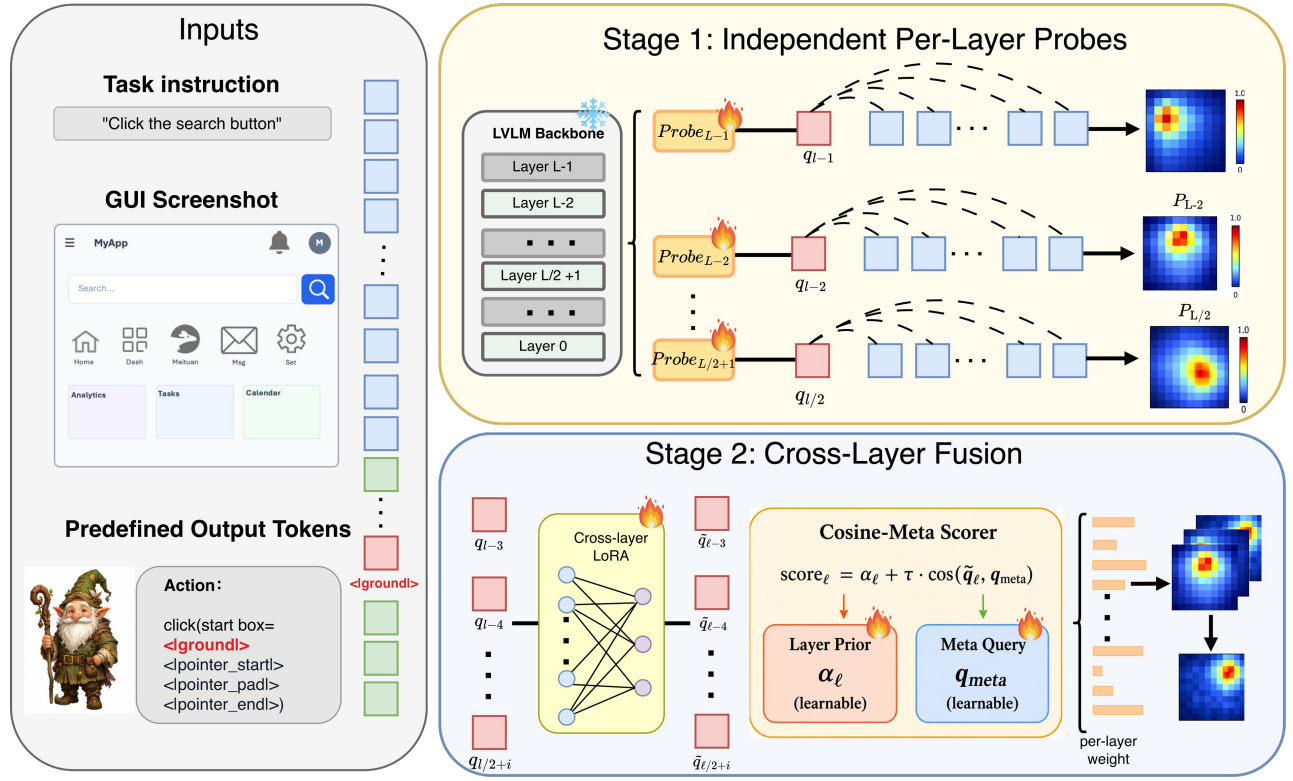


Figure 3: **Overview of ZwerGe-UI.** Given a GUI screenshot, instruction, and prefill anchor, a frozen backbone exposes layerwise hidden states; independent probes decode intermediate-layer spatial posteriors, which are dynamically weighted by a cross-layer LoRA and cosine-meta scorer to produce the final coordinate-free grounding heatmap.

fixed template ending in an anchor token:

$$\text{click}(\underbrace{\langle | \text{ground} | \rangle}_{\text{anchor}} \underbrace{\langle | \text{pointer_start} | \rangle \dots}_{\text{coord}})$$

and run a **single forward pass**. At the anchor position, the model has processed the full screenshot and instruction but no coordinate value, so there is zero label leakage.

RMSNorm pre-scale. Hidden-state ℓ_2 norms grow substantially with layer depth due to the geometrically misaligned activation spaces across layers. We apply a pre-scale $\mathbf{h} \leftarrow \mathbf{h} / (\|\mathbf{h}\|_2 / \sqrt{d})$ before each LayerNorm, bounding the input norm to $\approx \sqrt{d}$ regardless of depth and preventing numerical overflow in bfloat16. This also explains the heterogeneity of the semantic spaces handled by different layers from the perspective of activation norm.

Two training stages. Stage 1 trains one independent probe per layer with the backbone frozen, sufficient both for the probe experiments in §?? and as a standalone grounding system. Stage 2 loads the Stage 1 checkpoint and trains a lightweight cross-layer fusion head on top of the frozen probes.

Stage 1: Independent Per-Layer Probes

Setup. Let f_θ be a frozen backbone with L layers. Given input $\mathbf{x}$ (screenshot + instruction + prefill template), we extract per-layer hidden states via forward hooks. For each

probe layer $\ell \in \mathcal{L}_{\text{probe}}$ (the deeper half of the backbone, where spatial signal concentrates), we read the anchor query $\mathbf{h}_{t^*}^{(\ell)} \in \mathbb{R}^d$ at the $\langle | \text{ground} | \rangle$ position and visual keys $\mathbf{H}_v^{(\ell)} \in \mathbb{R}^{N_v \times d}$ over the N_v patch tokens. This corresponds to the layerwise branches in Figure ??, where each selected transformer layer independently maps the anchor token and visual patch tokens into a spatial heatmap over the screen.

Cross-attention probe. Each probe ϕ_ℓ computes a patch-level spatial posterior via multi-head cross-attention:

$$\begin{aligned} \mathbf{Q}_\ell &= W_q^{(\ell)} \text{Norm}(\mathbf{h}_{t^*}^{(\ell)}), \\ \mathbf{K}_\ell &= W_k^{(\ell)} \text{Norm}(\mathbf{H}_v^{(\ell)}), \end{aligned} \quad (1)$$

where $\text{Norm}(\cdot)$ denotes the RMS pre-scale followed by LayerNorm, and $W_q^{(\ell)}, W_k^{(\ell)}$ are full-rank projections (no bottleneck), letting each probe learn a fresh geometry suited to its layer. Let $\sigma(\cdot)$ denote the softmax operation. Per-head scores are combined via a learnable gate α :

$$p_\ell = \sigma \left(\sum_h \sigma(\alpha)_h \cdot \frac{\mathbf{Q}_{\ell,h}^\top \mathbf{K}_{\ell,h}}{\sqrt{d_h}} \right) \in \Delta^{N_v-1}. \quad (2)$$

Each probe contributes $\approx 4.2\text{M}$ parameters ($d=4096, n_h=8, d_h=64$), and the full probe set totals $\approx 42\text{M}$ parameters, or 0.52% of the 7B backbone (Qwen2.5-VL). For Qwen3-VL, this ratio is of the same order of magnitude.

Supervision. Each probe is supervised with an anisotropic Gaussian label centered at the bounding-box centroid. For patch i at position (x_i, y_i) :

$$y_i \propto \exp\left(-\frac{(x_i - c_x)^2}{2\sigma_x^2} - \frac{(y_i - c_y)^2}{2\sigma_y^2}\right), \quad (3)$$

with $\sigma_x = \eta w_b$, $\sigma_y = \eta h_b$, $\eta = 0.35$. This soft label reduces over-penalization from patch-grid quantization on small high-resolution targets (Finding 4), and the variance σ_x, σ_y is computed per sample and embedded inline rather than treated as a global constant. Stage 1 minimizes:

$$\mathcal{L}_{S1} = \frac{1}{|\mathcal{L}_{\text{probe}}|} \sum_{\ell} D_{\text{KL}}(\mathbf{y} \| p_{\ell}). \quad (4)$$

Gradients from ℓ affect only ϕ_{ℓ} ; the backbone receives zero gradient throughout.

Stage 2: Cross-Layer Fusion

Stage 1 produces independent per-layer posteriors whose quality varies non-monotonically across depth (Finding 1), and the optimal layer differs by sample. A static average is suboptimal. Stage 2 trains a cross-layer fusion head to *select and weight* per-layer posteriors conditioned on their content, with all backbone and probe weights frozen. As illustrated in the right part of Figure ??, these layerwise heatmaps are not averaged uniformly; instead, the fusion module assigns input-dependent weights to recover the most reliable spatial posterior.

Active probe layers. We use a subset $\mathcal{L}_{\text{active}} \subset \mathcal{L}_{\text{probe}}$ drawn from the spatial plateau identified in §?? as inputs to fusion; the remaining probe layers are retained in the checkpoint but not fused.

Fusion head. The fusion head processes anchor representations $\{\mathbf{h}_{t^*}^{(\ell)}\}_{\ell \in \mathcal{L}_{\text{active}}}$. A lightweight cross-layer LoRA module exchanges information across depths and produces refined queries $\hat{\mathbf{q}}_{\ell}$. A cosine-meta scorer then produces a per-layer weight:

$$p_{\text{final}} = \sum_{\ell} \sigma(\text{score})_{\ell} \cdot p_{\ell}. \quad (5)$$

The term α_{ℓ} is a sample-independent trainable prior capturing the average quality of layer ℓ across the training distribution. The cosine term computes a sample-dependent posterior: $\mathbf{q}_{\text{meta}}$ is a global trainable meta-query acting as a soft prototype for the ideal grounding representation, and τ is a learnable temperature; layers whose refined query aligns with $\mathbf{q}_{\text{meta}}$ receive higher weight for the current input. All fusion parameters initialize so that weights are uniform at the start of training. The cross-layer LoRA contributes $\approx 200\text{K}$ parameters in total, two orders of magnitude fewer than a single probe.

Stage 2 training. Only fusion parameters are updated. The loss combines a top-level KL term on p_{final} with a per-layer auxiliary:

$$\mathcal{L}_{S2} = D_{\text{KL}}(\mathbf{y} \| p_{\text{final}}) + \frac{\lambda}{|\mathcal{L}_{\text{active}}|} \sum_{\ell} D_{\text{KL}}(\mathbf{y} \| p_{\ell}), \quad (6)$$

Table 3: **ScreenSpot-v2**. Base rows: overlap@1. **+ZwerGe** rows: overlap@3.

Backbone	Sys.	Mob.	Desk.	Web	Ovr.
GUI-Owl-7B	Base (ov@1)	92.65	88.52	87.81	89.69
	+ZwerGe (ov@3)	96.88	96.72	91.49	94.65
UI-TARS-7B	Base (ov@1)	91.31	92.79	88.97	90.72
	+ZwerGe (ov@3)	95.32	96.39	92.65	94.49
Owl-1.5-8B	Base (ov@1)	95.77	94.10	92.84	94.18
	+ZwerGe (ov@3)	96.44	95.74	94.20	95.36
UI-Venus-8B	Base (ov@1)	97.33	98.03	95.74	96.85
	+ZwerGe (ov@3)	98.00	97.70	95.55	96.93

Table 4: **OSWorld-G** (refusal samples excluded). Baseline: overlap@1. **+ ZwerGe**: overlap@3.

Backbone	System	Text	Elem.	Layout	Fine-gr.	Overall
GUI-Owl-7B	Baseline (ov@1)	63.16	69.40	68.89	56.82	65.69
	+ ZwerGe (ov@3)	94.74	84.33	87.56	71.97	82.94
UI-TARS-1.5-7B	Baseline (ov@1)	57.89	70.90	77.33	62.88	71.18
	+ ZwerGe (ov@3)	84.21	83.58	82.22	75.00	80.78
GUI-Owl-1.5-8B	Baseline (ov@1)	78.95	80.60	80.89	81.82	80.98
	+ ZwerGe (ov@3)	73.68	88.81	88.89	75.76	84.90
UI-Venus-1.5-8B	Baseline (ov@1)	68.42	86.57	86.22	86.36	85.69
	+ ZwerGe (ov@3)	68.42	92.54	88.44	70.45	84.12

with $\lambda=0.2$. The auxiliary term prevents fusion from collapsing onto a single dominant layer by keeping all active probe posteriors individually calibrated.

Experiments

Experimental Setup

Benchmarks. We evaluate on five GUI grounding benchmarks spanning desktop, web, and mobile platforms. **ScreenSpot-Pro** (?) is a high-resolution desktop grounding benchmark with fine-grained domain categories (Development, Creative, CAD, Scientific, Office, OS); it is the primary benchmark for our analysis (§??) and main results. **ScreenSpot-v2** (?) covers mobile, desktop, and web screenshots at moderate resolution. **OSWorld-G** (?) provides GUI grounding samples drawn from real computer task executions, categorized by task type (Text Matching, Element Recognition, Layout Understanding, Fine-grained Manipulation). We remove refusal samples from the evaluation set and report results on the remaining non-refusal subset. **MMBench-GUI** (?) is a multi-platform benchmark covering Windows, macOS, Linux, iOS, Android, and Web. **UI-Vision** (?) evaluates grounding across Basic, Functional, Spatial, and Layout task types; we report the weighted average (Avg) over all 5,790 samples.

Metric. We report **overlap@k**: a click is counted as correct if a 1×1 -patch box centered at the predicted point (half-width 0.5 patch, i.e., 14–16 px depending on backbone patch size) intersects the ground-truth bounding box, taking the best of the top- k candidates. This differs from the standard **hit@1** metric (point strictly inside the box, single prediction) used on public leaderboards; overlap@ k is

Table 2: **ScreenSpot-Pro**. Baseline rows report overlap@1 (single autoregressive coordinate); + **ZwerGe** rows report overlap@3, taking the best of the top-3 patch-posterior regions from a single forward pass (see §??). Overall is the macro-average across six domains.

Backbone	System	Dev.	Creative	CAD	Scientific	Office	OS	Overall
GUI-Owl-7B	Baseline (ov@1)	49.50	44.28	50.96	60.63	66.96	42.86	52.12
	+ ZwerGe (ov@3)	68.56	65.69	64.75	71.65	81.30	62.76	68.94
UI-TARS-1.5-7B	Baseline (ov@1)	48.16	45.16	45.59	55.91	71.74	42.86	51.11
	+ ZwerGe (ov@3)	58.19	57.48	58.24	68.90	82.17	56.63	63.06
GUI-Owl-1.5-8B	Baseline (ov@1)	76.92	69.21	71.26	76.38	88.26	68.37	74.83
	+ ZwerGe (ov@3)	81.94	69.21	68.97	79.92	89.57	71.94	76.60
UI-Venus-1.5-8B	Baseline (ov@1)	70.90	60.12	67.82	71.26	87.83	69.90	70.46
	+ ZwerGe (ov@3)	78.60	67.74	70.11	74.02	88.70	75.51	75.21

strictly easier and the two numbers are *not* directly comparable. For autoregressive **baseline** models we report overlap@1: the model emits a single (x, y) coordinate via standard decoding, and we evaluate whether the resulting one-patch box touches the ground-truth box. For + **ZwerGe** we report overlap@3: our probe produces a full posterior over all visual-token patches in a single forward pass (no autoregressive decoding), from which we extract the top-3 connected-component peaks (§??) as three candidate points, and count a hit if any of the three overlaps the ground-truth box. Because all three candidates are read off the same forward pass, reporting overlap@3 does not add extra inference cost relative to overlap@1 for ZWERGE-UI (no additional decoding step is performed), but it is a different $\hat{A}$ and less strict $\hat{A}$ comparison than the single-candidate overlap@1 used for the baseline rows, and the two protocols should not be read as a matched ablation of the retrofit alone. We report both protocols explicitly, as-is, in every table below rather than normalizing them to a common metric.

Baselines. We retrofit ZWERGE-UI onto four GUI agents: **GUI-Owl-7B** (?) and **UI-TARS-1.5-7B** (?) (Qwen2.5-VL backbone, 28 layers) and **GUI-Owl-1.5-8B** (?) and **UI-Venus-1.5-8B** (?) (Qwen3-VL backbone, 36 layers). Each backbone is kept completely frozen throughout both training stages; only the probe and fusion heads are updated.

Implementation. Stage 1 (A7, §??) trains 14 $\hat{A}$ \$18 independent CrossAttn probes per model on 200k grounding samples for 1 epoch. Stage 2 (A8, §??) loads the Stage 1 checkpoint and trains only the Cross-Layer Fusion head (12 active layers, fusion_lora_rank=128, context_rank=64, $\lambda=0.2$) for 1 epoch on the same data. Inference uses the BFS centroid decode strategy with a $0.3 \times \max$ threshold; no autoregressive decoding is performed.

Main Results

Tables ?? $\hat{A}$ \$?? report overlap@1 for baseline rows and overlap@3 for ZWERGE-UI rows (§??), for all four backbone models. Each backbone occupies two adjacent rows: the original model (baseline, overlap@1) followed by its

Table 6: **UI-Vision**. Baseline: overlap@1. + **ZwerGe**: overlap@3. Avg is the weighted average over all 5,790 samples (Basic 1772 + Functional 1772 + Spatial 1935 + Layout 311).

Backbone	System	Basic	Functional	Spatial	Layout	Avg
GUI-Owl-7B	Baseline (ov@1)	33.07	34.37	11.52	64.31	27.94
	+ ZwerGe (ov@3)	75.56	69.86	46.25	70.42	63.75
UI-TARS-1.5-7B	Baseline (ov@1)	30.47	28.39	8.73	57.23	24.01
	+ ZwerGe (ov@3)	63.49	55.19	30.08	72.03	50.24
GUI-Owl-1.5-8B	Baseline (ov@1)	48.53	44.92	33.23	65.92	43.25
	+ ZwerGe (ov@3)	68.23	61.46	43.62	72.03	58.13
UI-Venus-1.5-8B	Baseline (ov@1)	59.82	55.81	40.41	64.31	52.35
	+ ZwerGe (ov@3)	67.16	63.37	46.36	80.06	59.74

ZWERGE-UI retrofitted version (+ **ZwerGe**, overlap@3). Because the two rows use different protocols, the absolute deltas discussed below conflate the retrofit itself with the relaxation from a single autoregressive prediction to the best of three patch-posterior candidates read from one forward pass, and should not be read as an ablation of the retrofit in isolation. Since ZWERGE-UI is an incremental retrofit rather than a standalone agent, we do not include external leaderboard entries in the main comparison $\hat{A}$ the primary question is whether retrofitting a frozen backbone with our grounding heads provides a consistent improvement under this protocol.

Qwen2.5-VL backbones benefit more than Qwen3-VL backbones. On ScreenSpot-Pro, GUI-Owl-7B (Qwen2.5-VL) gains +16.82 pp overall (52.12 $\rightarrow$ 68.94) and UI-TARS-1.5-7B gains +11.95 pp (51.11 $\rightarrow$ 63.06); GUI-Owl-1.5-8B (Qwen3-VL) gains only +1.77 pp (74.83 $\rightarrow$ 76.60) and UI-Venus-1.5-8B gains +4.75 pp (70.46 $\rightarrow$ 75.21). This pattern is consistent with the our hypothesis: Qwen3-VL models are trained on substantially more diverse grounding supervision, yielding stronger end-to-end coordinate serialization and a smaller gap between intermediate spatial quality and final-layer output. When the backbone already converts spatial intent to coordinates reliably, bypassing the final-layer decoder recovers less headroom, even after the top-3 relaxation.

Table 5: **MMBench-GUI**. Baseline: overlap@1. + **ZwerGe**: overlap@3.

Backbone	System	Windows	macOS	Linux	iOS	Android	Web	Overall
GUI-Owl-7B	Baseline (ov@1)	61.69	66.86	55.81	83.54	82.70	77.51	72.84
	+ ZwerGe (ov@3)	82.69	87.12	78.29	91.93	92.97	89.32	87.90
UI-TARS-1.5-7B	Baseline (ov@1)	64.64	71.20	61.24	84.01	85.94	80.74	75.99
	+ ZwerGe (ov@3)	78.27	84.08	71.58	92.86	92.83	85.92	85.48
GUI-Owl-1.5-8B	Baseline (ov@1)	79.37	82.92	66.67	89.60	88.47	86.25	83.50
	+ ZwerGe (ov@3)	82.50	88.42	80.10	93.79	94.80	90.13	89.15
UI-Venus-1.5-8B	Baseline (ov@1)	84.90	87.41	76.49	93.48	94.51	91.10	88.98
	+ ZwerGe (ov@3)	85.82	91.61	83.20	93.94	97.33	91.10	91.29

Table 7: **Ablation** on UI-TARS-1.5-7B (overlap@1, %, checkpoint-800). Rows A0–A5 vary GT label type, fusion scorer, and probe layer range. **GT**: Binary vs. Gaussian. **Fusion**: Readiness-MLP vs. CosMeta. **Layers**: probe layer range (all within L18–L27).

ID	GT	Fusion	Layers	SS-Pro	SS-v2	OSW-G
A0	Binary	Readiness	L18–27	46.4	89.3	72.0
A1	Gaussian	Readiness	L18–27	47.1	90.0	57.6
A2	Binary	CosMeta	L18–27	49.6	90.6	59.4
A3	Gaussian	CosMeta	L18–27	45.2	89.7	57.6
A4	Gaussian	CosMeta	L18–25	48.7	92.0	59.2
A5	Gaussian	CosMeta	L18–23	38.4	87.6	49.4

High-resolution layouts remain a challenge for patch-level decoding. ScreenSpot-Pro is the most demanding benchmark because targets are often single-pixel or sub-icon elements on $2,560 \times 1,440$ or larger screenshots. ZWERGE-UI replaces pixel-precise coordinate regression with a patch-level posterior over the visual token grid, so the minimum localization granularity is one patch rather than a single pixel. On ScreenSpot-Pro, several domain categories (CAD, Creative) show the smallest gains even under the top-3 relaxation, whereas moderate-resolution benchmarks (ScreenSpot-v2, MMBench-GUI) are more uniformly positive. The patch granularity is an architectural trade-off: it enables frozen-backbone readout without any coordinate vocabulary but introduces a resolution ceiling that limits gains on the hardest high-resolution domains, a ceiling that additional candidates alone cannot fully remove.

Spatial grounding tasks benefit most. UI-Vision decomposes grounding into Basic, Functional, Spatial, and Layout subtypes. Baseline models are weakest on Spatial (8.73–40.41%), and ZWERGE-UI provides the largest absolute improvements there: GUI-Owl-7B improves Spatial from 11.52 to 46.25% (+34.73 pp) and UI-TARS-1.5-7B from 8.73 to 30.08% (+21.35 pp). This is the task type most directly probing whether a model has formed a spatial posterior over the screen, which is precisely the signal read by our intermediate-layer probes.

Ablations

Setup. We ablate three design choices on UI-TARS-1.5-7B using a fast LoRA-probe variant, separate from the final CrossAttn probes in §???. All variants are trained on 50k samples and evaluated at checkpoint 800. We vary: (i) **GT label type**, binary box labels vs. anisotropic Gaussian labels; (ii) **fusion scorer**, Readiness-MLP vs. CosMeta from §??; and (iii) **probe layer range**, varied within L18–L27.

Results. Table ?? shows three trends. First, fusion matters most on SS-Pro: replacing Readiness-MLP with CosMeta improves Binary GT from A0 to A2 by +3.2 pp, indicating that sample-dependent cosine weighting is more effective than hand-crafted readiness features. Second, GT label type interacts with fusion: Gaussian labels slightly help under Readiness (A0→A1) but lag behind Binary under CosMeta at this early checkpoint (A2 vs. A3), suggesting soft targets need longer convergence or different fusion calibration. Third, layer range is critical. Removing the deepest layers L26–L27 improves SS-Pro by +3.5 pp (A3→A4), consistent with our finding that late layers are more serialization-oriented than spatially informative. Truncating further to L18–L23 sharply hurts performance (A4→A5), identifying L18–L25 as the best trade-off band.

Conclusion

We present ZWERGE-UI, a coordinate-free retrofit that exposes and exploits the coordinate serialization bottleneck in GUI agents. Layerwise probing shows that instruction-conditioned spatial posteriors often peak at intermediate transformer layers, while later layers increasingly specialize in coordinate serialization rather than localization refinement. Motivated by this finding, ZWERGE-UI freezes the original backbone and attaches lightweight per-layer grounding probes with a cross-layer fusion head, enabling single-pass spatial readout without autoregressive coordinate generation. Across diverse GUI grounding benchmarks, the retrofit recovers useful intermediate-layer spatial signals, with the strongest gains on spatial and layout-centric tasks. The remaining failures reveal two limitations: patch-level granularity on high-resolution small targets and OCR-dominated text matching. Overall, our results suggest that future GUI agents should decouple spatial ground-

ing from coordinate serialization and combine intermediate-layer spatial evidence with finer-grained visual and textual signals.

References

- Abnar, S.; and Zuidema, W. 2020. Quantifying Attention Flow in Transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4190–4197.
- Bai, J.; Bai, S.; Yang, S.; Wang, S.; Tan, S.; Wang, P.; Lin, J.; Zhou, C.; and Zhou, J. 2023. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. *arXiv preprint arXiv:2308.12966*.
- Bai, S.; Chen, K.; Liu, X.; Wang, J.; Ge, W.; Song, S.; Dang, K.; Wang, P.; Wang, S.; Tang, J.; et al. 2025. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923*.
- Cao, Q.; Paranjape, B.; and Hajishirzi, H. 2023. PuMer: Pruning and Merging Tokens for Efficient Vision Language Models. *arXiv preprint arXiv:2305.17530*.
- Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; and Zagoruyko, S. 2020. End-to-End Object Detection with Transformers. In *European Conference on Computer Vision*, 213–229.
- Chefer, H.; Gur, S.; and Wolf, L. 2021. Transformer Interpretability Beyond Attention Visualization. In *CVPR*, 782–791.
- Chen, H.; Lin, J.; Chen, X.; Fan, Y.; Jin, X.; Su, H.; Dong, J.; Fu, J.; and Shen, X. 2025. Rethinking visual layer selection in multimodal llms. *arXiv e-prints*, arXiv–2504.
- Chen, K.; Zhang, Z.; Zeng, W.; Zhang, R.; Zhu, F.; and Zhao, R. 2023. Shikra: Unleashing Multimodal LLM’s Referential Dialogue Magic. *arXiv preprint arXiv:2306.15195*.
- Chen, L.; Zhao, H.; Liu, T.; Bai, S.; Lin, J.; Zhou, C.; and Chang, B. 2024. An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models. In *European Conference on Computer Vision*.
- Cheng, K.; Sun, Q.; Chu, Y.; Xu, F.; YanTao, L.; Zhang, J.; and Wu, Z. 2024. SeeClick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 9313–9332.
- Darcet, T.; Oquab, M.; Mairal, J.; and Bojanowski, P. 2024. Vision Transformers Need Registers. In *ICLR*.
- Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S.; et al. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *ICLR*.
- Fan, Y.; Ding, L.; Kuo, C.-C.; Jiang, S.; Zhao, Y.; Guan, X.; Yang, J.; Zhang, Y.; and Wang, X. E. 2024. Read Anywhere Pointed: Layout-aware GUI Screen Reading with Tree-of-Lens Grounding. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 9503–9522. Miami, Florida, USA: Association for Computational Linguistics.
- Gou, B.; Wang, R.; Zheng, B.; Xie, Y.; Chang, C.; Shu, Y.; Sun, H.; and Su, Y. 2025. Navigating the digital world as humans do: Universal visual grounding for gui agents. In *ICLR*.
- Gu, Z.; Zeng, Z.; Xu, Z.; Zhou, X.; Shen, S.; Liu, Y.; Zhou, B.; Meng, C.; Xia, T.; Chen, W.; et al. 2025. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*.
- Hong, W.; Wang, W.; Lv, Q.; Xu, J.; Yu, W.; Ji, J.; Wang, Y.; Wang, Z.; Dong, Y.; Ding, M.; et al. 2024. Cogagent: A visual language model for gui agents. In *CVPR*, 14281–14290.
- Kamath, A.; Singh, M.; LeCun, Y.; Synnaeve, G.; Misra, I.; and Carion, N. 2021. MDETR: Modulated Detection for End-to-End Multi-Modal Understanding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 1780–1790.
- Kang, S.; Kim, J.; Kim, J.; and Hwang, S. J. 2025. See What You Are Told: Visual Attention Sink in Large Multimodal Models. In *ICLR*.
- Kim, W.; Son, B.; and Kim, I. 2021. ViLT: Vision-and-Language Transformer Without Convolution or Region Supervision. In *International Conference on Machine Learning*, 5583–5594.
- Kirillov, A.; Mintun, E.; Ravi, N.; Mao, H.; Rolland, C.; Gustafson, L.; Xiao, T.; Whitehead, S.; Berg, A. C.; Lo, W.-Y.; et al. 2023. Segment Anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 4015–4026.
- Lan, M.; Chen, C.; Ke, Y.; Wang, X.; Feng, L.; and Zhang, W. 2024. Clearclip: Decomposing clip representations for dense vision-language inference. In *ECCV*, 143–160. Springer.
- Li, K.; Meng, Z.; Lin, H.; Luo, Z.; Tian, Y.; Ma, J.; Huang, Z.; and Chua, T.-S. 2025. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *ACM MM*, 8778–8786.
- Li, L. H.; Zhang, P.; Zhang, H.; Yang, J.; Li, C.; Zhong, Y.; Wang, L.; Yuan, L.; Zhang, L.; Hwang, J.-N.; et al. 2022. Grounded Language-Image Pre-training. In *CVPR*, 10965–10975.
- Li, W.; Yuan, Y.; Liu, J.; Tang, D.; Wang, S.; Qin, J.; Zhu, J.; and Zhang, L. 2024. TokenPacker: Efficient Visual Projector for Multimodal LLM. *arXiv preprint arXiv:2407.02392*.
- Lin, J.; Shen, F.; Li, J.; Nie, P.; Yu, F.; Li, M.; and Li, H. 2026. What Happens Before Decoding? Prefill Determines GUI Grounding in VLMs. *arXiv preprint arXiv:2605.12549*.
- Lin, K. Q.; Li, L.; Gao, D.; Yang, Z.; Wu, S.; Bai, Z.; Lei, S. W.; Wang, L.; and Shou, M. Z. 2025. Showui: One vision-language-action model for gui visual agent. In *CVPR*, 19498–19508.
- Liu, S.; Zeng, Z.; Ren, T.; Li, F.; Zhang, H.; Yang, J.; Jiang, Q.; Li, C.; Yang, J.; Su, H.; et al. 2024. Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. In *European Conference on Computer Vision*.

- Liu, Y.; Li, P.; Xie, C.; Hu, X.; Han, X.; Zhang, S.; Yang, H.; and Wu, F. 2025. Infigui-r1: Advancing multimodal gui agents from reactive actors to deliberative reasoners. *arXiv preprint arXiv:2504.14239*.
- Liu, Y.; Liu, Z.; Zhu, S.; Li, P.; Xie, C.; Wang, J.; Hu, X.; Han, X.; Yuan, J.; Wang, X.; et al. 2026. Infigui-gl: Advancing gui grounding with adaptive exploration policy optimization. In *AAAI*, 38, 32267–32275.
- Lu, Y.; Yang, J.; Shen, Y.; and Awadallah, A. 2024. Omniparser for pure vision based gui agent. *arXiv preprint arXiv:2408.00203*.
- Lu, Z.; Chai, Y.; Guo, Y.; Yin, X.; Liu, L.; Wang, H.; Xiao, H.; Ren, S.; Xiong, G.; and Li, H. 2025. UI-R1: Enhancing Efficient Action Prediction of GUI Agents by Reinforcement Learning. *arXiv preprint arXiv:2503.21620*.
- Luo, R.; Wang, L.; He, W.; Chen, L.; Li, J.; and Xia, X. 2025a. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*.
- Luo, T.; Logeswaran, L.; Johnson, J.; and Lee, H. 2025b. Visual Test-time Scaling for GUI Agent Grounding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*.
- Minderer, M.; Gritsenko, A.; Stone, A.; Neumann, M.; Weissenborn, D.; Dosovitskiy, A.; Mahendran, A.; Arnab, A.; Dehghani, M.; Shen, Z.; et al. 2022. Simple Open-Vocabulary Object Detection with Vision Transformers. In *European Conference on Computer Vision*.
- Nayak, S.; Jian, X.; Lin, K. Q.; Rodriguez, J. A.; Kalsi, M.; Awal, R.; Chapados, N.; Özsu, M. T.; Agrawal, A.; Vazquez, D.; et al. 2025. Ui-vision: A desktop-centric gui benchmark for visual perception and interaction. In *ICML*.
- Ouyang, M.; Lin, K. Q.; Shou, M. Z.; and Ng, H. T. 2026. FocusUI: Efficient UI Grounding via Position-Preserving Visual Token Selection. *arXiv preprint arXiv:2601.03928*.
- Peng, Z.; Wang, W.; Dong, L.; Hao, Y.; Huang, S.; Ma, S.; and Wei, F. 2023. Kosmos-2: Grounding Multimodal Large Language Models to the World. *arXiv preprint arXiv:2306.14824*.
- Qin, Y.; Ye, Y.; Fang, J.; Wang, H.; Liang, S.; Tian, S.; Zhang, J.; Li, J.; Li, Y.; Huang, S.; et al. 2025. UI-TARS: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*.
- Qwen Team. 2025a. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*.
- Qwen Team. 2025b. Qwen3-VL Technical Report. *arXiv preprint arXiv:2511.21631*.
- Radford, A.; Kim, J. W.; Hallacy, C.; Ramesh, A.; Goh, G.; Agarwal, S.; Sastry, G.; Askell, A.; Mishkin, P.; Clark, J.; et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*, 8748–8763.
- Sun, L.; Chen, X.; Chen, L.; Dai, T.; Zhu, Z.; and Yu, K. 2022. META-GUI: Towards Multi-modal Conversational Agents on Mobile GUI. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 6699–6712. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics.
- Team, V.; Gao, C.; Gu, Z.; Liu, Y.; Qiu, X.; Shen, S.; Wen, Y.; Xia, T.; Xu, Z.; Zeng, Z.; et al. 2026. Ui-venus-1.5 technical report. *arXiv preprint arXiv:2602.09082*.
- Wang, X.; Wu, Z.; Xie, J.; Ding, Z.; Yang, B.; Li, Z.; Liu, Z.; Li, Q.; Dong, X.; Chen, Z.; et al. 2025. Mmbench-gui: Hierarchical multi-platform evaluation framework for gui agents. *arXiv preprint arXiv:2507.19478*.
- Wu, H.; Chen, H.; Cai, Y.; Liu, C.; Ye, Q.; Yang, M.-H.; and Wang, Y. 2025a. DiMo-GUI: Advancing Test-time Scaling in GUI Grounding via Modality-Aware Visual Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 26246–26256.
- Wu, Q.; Cheng, K.; Yang, R.; Zhang, C.; Yang, J.; Jiang, H.; Mu, J.; Peng, B.; Qiao, B.; Tan, R.; et al. 2025b. Gui-actor: Coordinate-free visual grounding for gui agents. In *NeurIPS*.
- Wu, Z.; Wu, Z.; Xu, F.; Wang, Y.; Sun, Q.; Jia, C.; Cheng, K.; Ding, Z.; Chen, L.; Liang, P. P.; et al. 2025c. Os-atlas: A foundation action model for generalist gui agents. In *ICLR*.
- Wu, Z.; Wu, Z.; Xu, F.; Wang, Y.; Sun, Q.; Jia, C.; Cheng, K.; Ding, Z.; Chen, L.; Liang, P. P.; et al. 2025d. OS-ATLAS: Foundation Action Model for Generalist GUI Agents. In *ICLR*.
- Xiao, G.; Tian, Y.; Chen, B.; Han, S.; and Lewis, M. 2024. Efficient Streaming Language Models with Attention Sinks. In *ICLR*.
- Xie, T.; Deng, J.; Li, X.; Yang, J.; Wu, H.; Chen, J.; Hu, W.; Wang, X.; Xu, Y.; Wang, Z.; et al. 2025. Scaling computer-use grounding via user interface decomposition and synthesis. In *NeurIPS*.
- Xu, H.; Zhang, X.; Liu, H.; Wang, J.; Zhu, Z.; Zhou, S.; Hu, X.; Gao, F.; Cao, J.; Wang, Z.; et al. 2026. Mobile-agent-v3.5: Multi-platform fundamental gui agents. *arXiv preprint arXiv:2602.16855*.
- Xu, H.-M.; Chen, Q.; Wang, L.; and Liu, L. 2025a. Attention-driven gui grounding: Leveraging pretrained multimodal large language models without fine-tuning. In *AAAI*, volume 39, 8851–8859.
- Xu, Y.; Wang, Z.; Wang, J.; Lu, D.; Xie, T.; Saha, A.; Sahoo, D.; Yu, T.; and Xiong, C. 2025b. Aguis: Unified pure vision agents for autonomous gui interaction. *ICML*.
- Yang, R.; Wu, Q.; Wang, Z.; Chen, H.; Yang, K.; Cheng, H.; Yao, H.; Peng, B.; Zhang, H.; Gao, J.; et al. 2026. Gui-libra: Training native gui agents to reason and act with action-aware supervision and partially verifiable rl. *arXiv preprint arXiv:2602.22190*.
- Ye, J.; Zhang, X.; Xu, H.; Liu, H.; Wang, J.; Zhu, Z.; Zheng, Z.; Gao, F.; Cao, J.; Lu, Z.; et al. 2025. Mobile-Agent-V3: Fundamental Agents for GUI Automation. *arXiv preprint arXiv:2508.15144*.
- You, H.; Zhang, H.; Gan, Z.; Du, X.; Zhang, B.; Wang, Z.; Cao, L.; Chang, S.-F.; and Yang, Y. 2024a. Ferret: Refer and Ground Anything Anywhere at Any Granularity. In *ICLR*.
- You, K.; Zhang, H.; Schoop, E.; Weers, F.; Swearngin, A.; Nichols, J.; Yang, Y.; and Gan, Z. 2024b. Ferret-UI: Grounded Mobile UI Understanding with Multimodal LLMs. In *European Conference on Computer Vision*.

Zhang, L.; Gao, L.; and Xu, M. 2025. Does chain-of-thought reasoning help mobile gui agent? an empirical study. *arXiv preprint arXiv:2503.16788*.

Zhang, L.; Xiao, Y.; Lu, X.; Cao, J.; Zhao, Y.; Zhou, J.; An, L.; Feng, Z.; Sha, W.; Shi, Y.; et al. 2026. OmegaUse: Building a General-Purpose GUI Agent for Autonomous Task Execution. *arXiv preprint arXiv:2601.20380*.

Zhou, Y.; Dai, S.; Wang, S.; Zhou, K.; Jia, Q.; and Xu, J. 2025. Gui-g1: Understanding r1-zero-like training for visual grounding in gui agents. In *NeurIPS*.

Related Work

GUI Agent Grounding

GUI grounding maps user instructions to executable screen elements or locations, serving as a core perception capability for GUI agents. Compared with general visual grounding, GUI grounding faces unique challenges from dense text, small icons, regular layouts, high-resolution screenshots, and platform-specific interaction semantics. Early GUI-oriented agents and datasets, such as META-GUI (?), connect task-oriented dialogue with real mobile GUI operations, showing that conversational agents can execute tasks on real apps without relying on task-specific backend APIs. Representative screenshot-based GUI agents and UI-centric MLLMs, such as CogAgent (?), SeeClick (?), Ferret-UI (?), and OmniParser (?), demonstrate that visual grounding can reduce reliance on HTML, DOM, or accessibility trees. Recent works further improve GUI grounding by scaling model capacity, GUI data, and action traces, including UGround (?), OS-Atlas (?), ShowUI (?), Aguvis (?), and UI-TARS (?). Meanwhile, ScreenSpot-Pro (?) shows that professional high-resolution GUI grounding remains challenging, especially for small targets, complex layouts, and fine-grained manipulation.

Most existing GUI grounding methods formulate localization as coordinate or action-parameter prediction. To improve this paradigm, recent works introduce GUI-specific reinforcement or reward signals, such as UI-R1 (?), GUI-R1 (?), InfiGUI-R1 (?), GUI-G1 (?), and InfiGUI-G1 (?). However, coordinate-based grounding may suffer from unstable numerical prediction and ambiguous point-level supervision, motivating coordinate-free, attention-based, or region-centric alternatives. Complementary to direct coordinate prediction, layout-aware GUI referring methods such as Tree-of-Lens (?) construct hierarchical layout structures from screenshots and user-indicated points, highlighting the importance of region semantics and spatial relationships for interpretable GUI understanding. Along this line, TAG (?), GUI-Actor (?), RegionFocus (?), DiMo-GUI (?), and FocusUI (?) explore intrinsic attention mining, action-region alignment, visual test-time scaling, modality-aware refinement, or position-preserving token selection. Concurrently, Re-Prefill (?) shows that the prefill stage largely determines candidate UI elements before coordinate decoding, and improves grounding via an attention-guided second prefill. This work shares our motivation that GUI grounding should not be viewed merely as autoregressive coordinate gener-

ation. However, Re-Prefill relies on raw cross-layer attention to construct target hypotheses and revisits inference with an additional prefill stage, whereas our method learns a lightweight frozen-backbone grounding head to explicitly compute token-region similarity and dynamically fuse task-relevant cues, providing a more direct, calibrated, and interpretable coordinate-free readout.

Visual Transformers Grounding

Visual Transformers provide the architectural foundation for modeling language, vision, and spatial correspondence in modern grounding systems. ViT (?) introduces patch-level visual tokenization, while CLIP (?) aligns visual and textual representations through large-scale contrastive pretraining. Building on these foundations, DETR (?), MDETR (?), GLIP (?), Grounding DINO (?), OWL-ViT (?), and SAM (?) extend detection toward phrase grounding, open-vocabulary localization, and promptable segmentation. More recently, grounded MLLMs such as Kosmos-2 (?), Shikra (?), Ferret (?), Qwen-VL (?), and Qwen2.5-VL (?) unify region understanding, coordinate prediction, and natural language generation, making grounding a general capability of multimodal instruction-following models. Despite their strong grounding ability, Transformer-based VLMs face efficiency and explainability challenges. On the efficiency side, high-resolution images introduce many visual tokens and substantially increase attention cost; therefore, efficient VLM and token compression methods such as ViLT (?), PuMer (?), FastV (?), and TokenPacker (?) reduce visual redundancy through lightweight encoders, pruning, merging, or compressed visual projectors. On the explainability side, Attention Rollout (?) and Transformer relevance propagation (?) attempt to identify visual evidence behind Transformer predictions. However, attention is not always a faithful explanation: attention sink in LLMs (?), register tokens in ViTs (?), and visual attention sink in LMMs (?) show that high-attention or 502(h-305(osh)25(w)-503(that)-6 Tf 4.9n)-298battenti2(loc

Per-Layer Grounding Accuracy: Full Curves

Figure ?? shows the per-layer overlap@1 of every A7 CrossAttn probe evaluated independently (no fusion), plotted against the transformer layer index, for all four backbone models and all five benchmarks. Two training checkpoints are compared: **ckpt-800** (dashed blue, early training) and the **last checkpoint** (solid red, ckpt-3129 for Qwen2.5-VL models and ckpt-3130 for Qwen3-VL models). Horizontal dotted lines mark the corresponding fusion head accuracy.

Setup. Each probe is a CrossAttn grounding head attached to a single transformer layer; all probes are trained simultaneously with independent per-layer KL losses (§??). At evaluation, each probe predicts independently the fusion head (dotted line) combines all active probes and serves as the reference ceiling. Backbone parameters are completely frozen throughout.

Observations.

- **Inverted-U spatial profile.** All four models exhibit a consistent inverted-U shape: per-layer accuracy rises from the first probed layer, peaks around the middle of the probed range (L20–L22 for 28-layer models; L22–L25 for 36-layer models), then declines toward the final layers. This is direct evidence for the coordinate serialization bottleneck described in §?: the deepest layers progressively encode coordinate formatting rather than spatial content.
- **Training improves mid-layer probes more than shallow or deep ones.** Comparing ckpt-800 (blue) to the last checkpoint (red), the largest absolute gains appear in the middle of the probed range. Shallow probes improve less because spatial features at early layers are coarser; deep probes improve less because the layers are increasingly occupied by serialization signal that the probe cannot redirect.
- **Fusion consistently outperforms the best single layer.** The fusion dotted line lies above the peak of the per-layer curve in nearly all panels. This confirms that the learned cross-layer aggregation in CONTEXTLoRACos-METAfusion recovers information not captured by any individual probe.
- **Qwen3-VL probes cover a wider band.** GUI-Owl-1.5-8B and UI-Venus-1.5-8B (36-layer Qwen3-VL, probes at L18–L35) show a broader and flatter peak compared to the 28-layer models (probes at L14–L27). The wider effective range suggests that the spatial-to-serialization transition is more gradual in deeper networks.

Figures ??–?? show the same data broken out by benchmark, at a larger scale for readability.

A7 CrossAttn Probe: Per-Layer overlap@1 (dashed=ckpt-800, solid=last ckpt)
Dotted horizontal lines show the fusion head accuracy

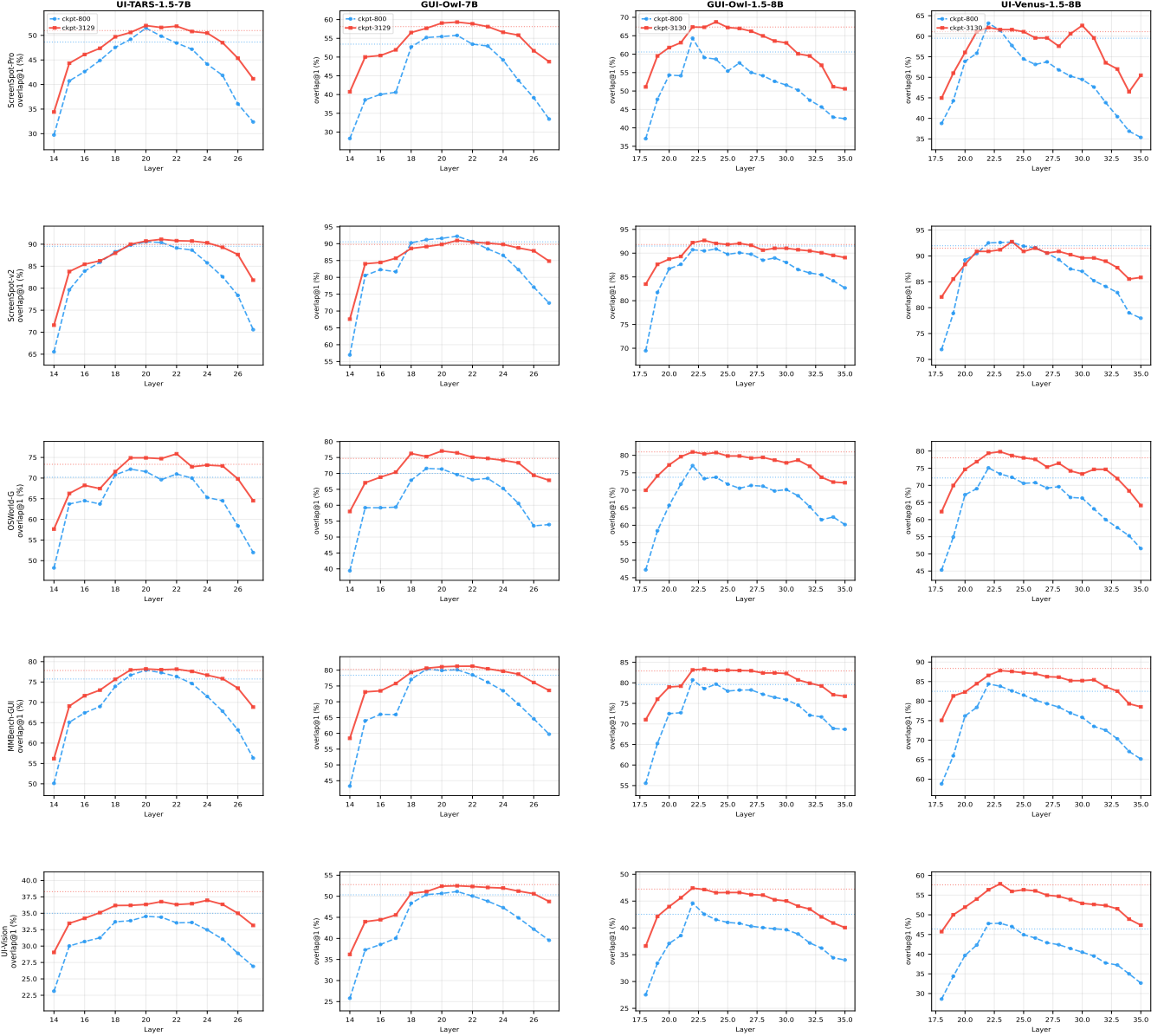


Figure 4: Per-layer overlap@1 of A7 CrossAttn probes across all four backbone models and five benchmarks. Each cell shows overlap@1 for a single probe layer evaluated independently (no cross-layer fusion). **Rows:** ScreenSpot-Pro (SS-Pro), ScreenSpot-v2 (SS-v2), OSWorld-G, MMBench-GUI, UI-Vision. **Columns:** UI-TARS-1.5-7B, GUI-Owl-7B, GUI-Owl-1.5-8B, UI-Venus-1.5-8B. **Blue dashed:** ckpt-800 (early training); **red solid:** last checkpoint (ckpt-3129 / ckpt-3130). **Horizontal dotted lines:** corresponding fusion head accuracy (the final ZWERGE-UI output). All four models exhibit an inverted-U spatial profile: accuracy peaks at intermediate layers and declines at the deepest layers, consistent with the coordinate serialization bottleneck (§??). The fusion head (dotted) consistently exceeds the best single-layer probe, confirming the benefit of learned cross-layer aggregation.

Per-Layer overlap@1 on ScreenSpot-Pro
(A7 CrossAttn Probe, each probe layer evaluated independently)

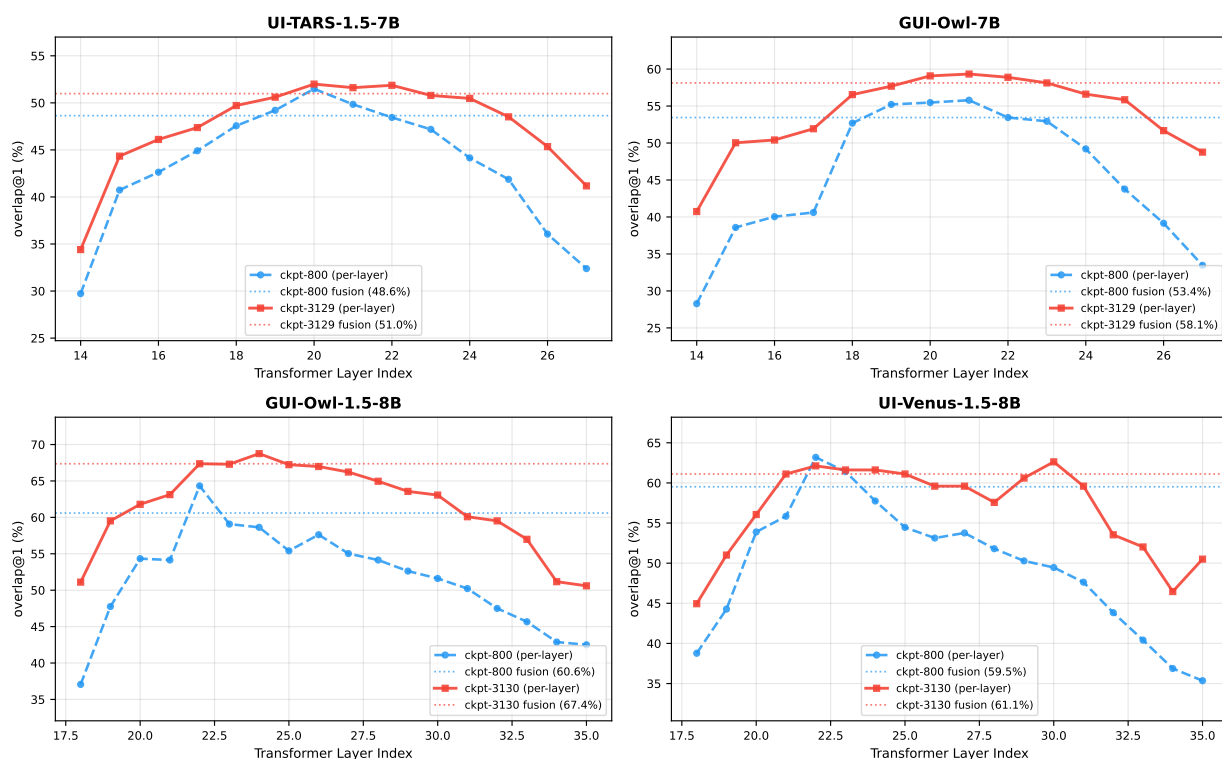


Figure 5: **Per-layer overlap@1 on ScreenSpot-Pro.** ScreenSpot-Pro is the most demanding benchmark (fine-grained targets on high-resolution desktop screenshots). The inverted-U profile is the sharpest here, with the deepest layers showing the steepest decline, reflecting that high-resolution targets require the most precise spatial encoding and are most sensitive to the serialization shift in deep layers. Note that the vertical scales differ across panels to emphasize the shape of the curve within each model.

Per-Layer overlap@1 on ScreenSpot-v2
(A7 CrossAttn Probe, each probe layer evaluated independently)

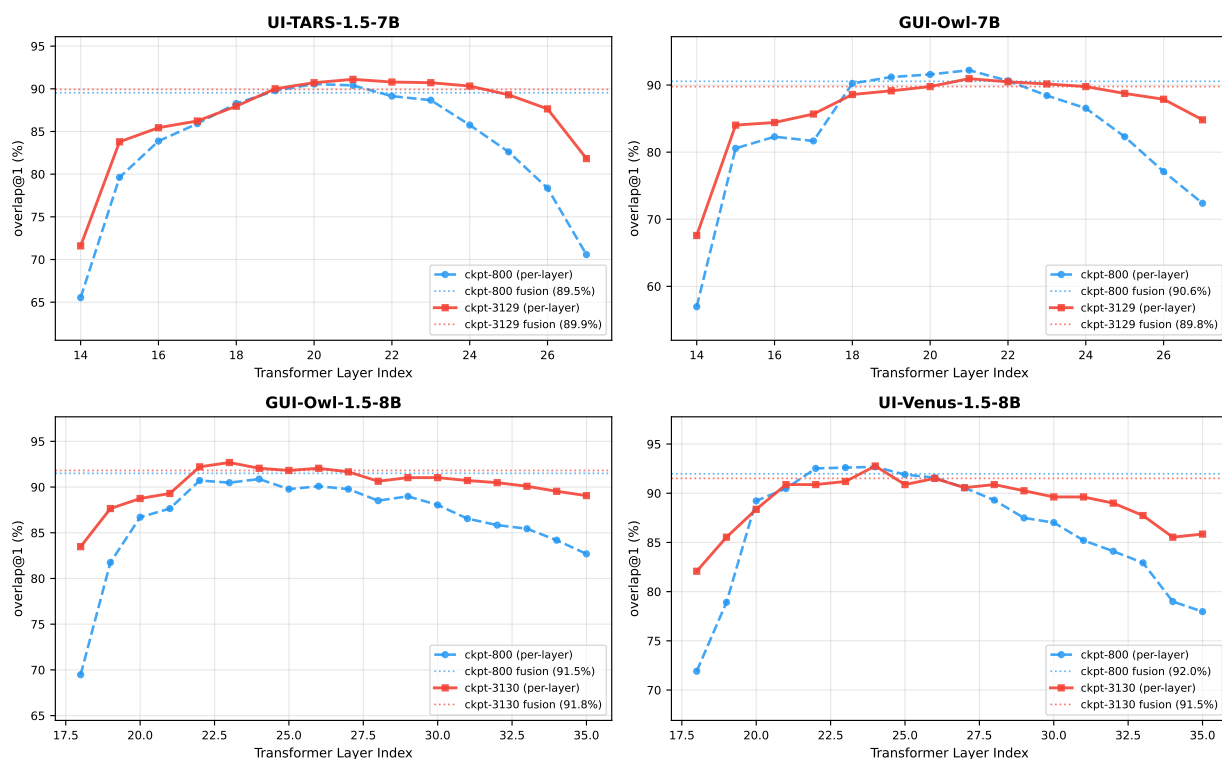


Figure 6: **Per-layer overlap@1 on ScreenSpot-v2.** ScreenSpot-v2 is a moderate-resolution benchmark (mobile, desktop, web) where overall accuracy is already high ($>87\%$ at ckpt-800). The inverted-U is shallower and the peak is broader compared to ScreenSpot-Pro, indicating that moderate-resolution grounding is less sensitive to which specific layer is read.

Per-Layer overlap@1 on OSWorld-G
(A7 CrossAttn Probe, each probe layer evaluated independently)

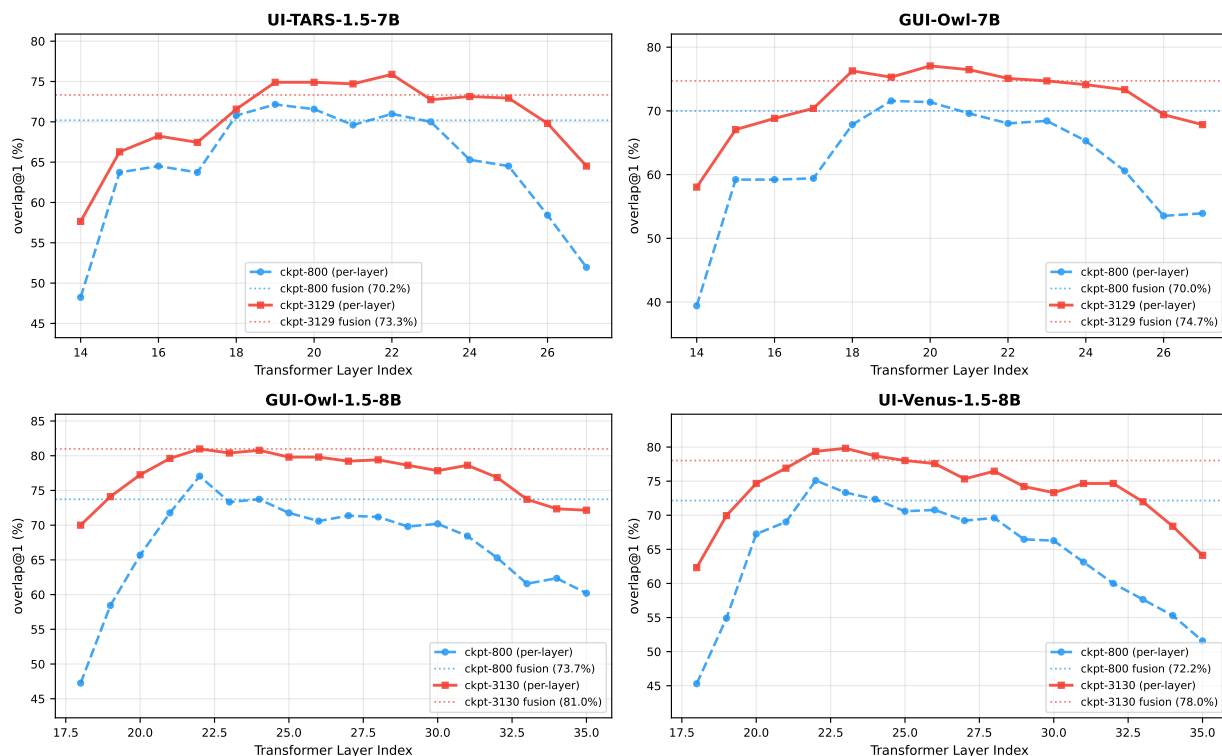


Figure 7: **Per-layer overlap@1 on OSWorld-G.** OSWorld-G provides real computer-task GUI samples with diverse task types (Text Matching, Element Recognition, Layout Understanding, Fine-grained Manipulation). The per-layer profile shows a similar inverted-U shape but with a plateau over a wider middle range ($\sim$ L18–L24 for 28-layer models), suggesting that layout-level spatial reasoning is distributed more broadly across intermediate layers than fine-grained element targeting.

Per-Layer overlap@1 on MMBench-GUI
(A7 CrossAttn Probe, each probe layer evaluated independently)

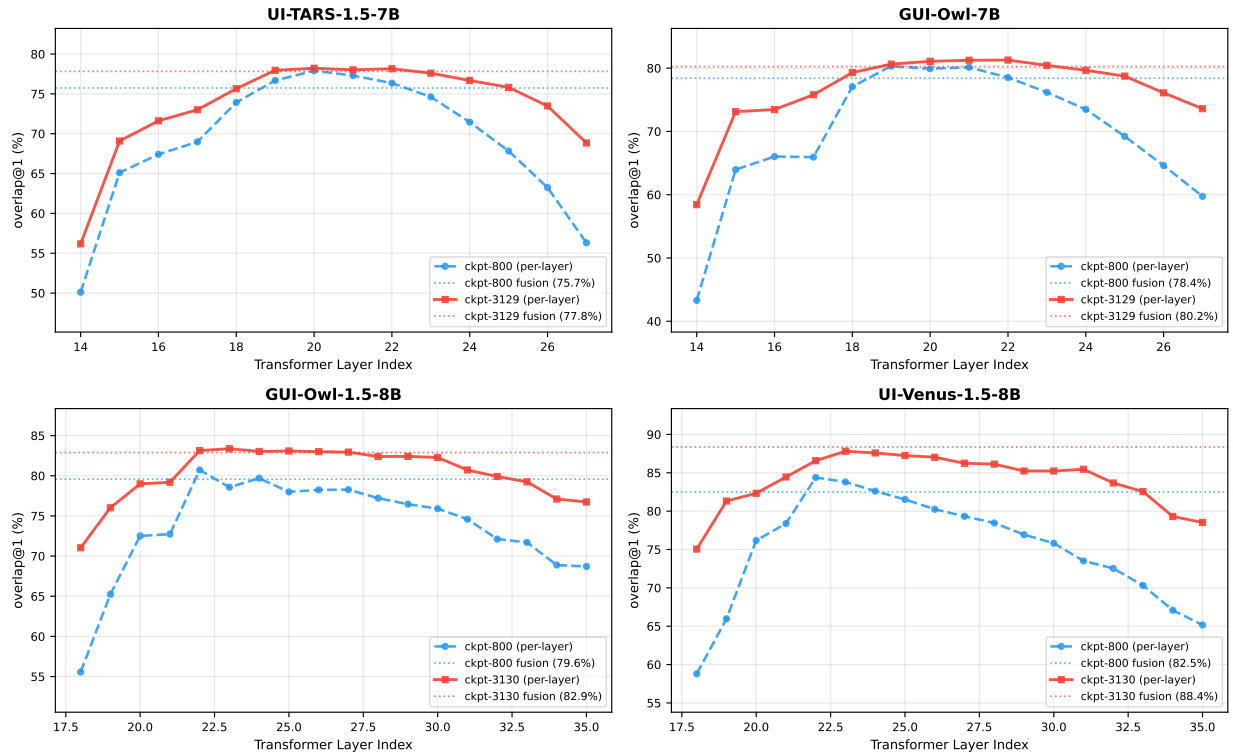


Figure 8: **Per-layer overlap@1 on MMBench-GUI.** MMBench-GUI covers six platforms (Windows, macOS, Linux, iOS, Android, Web). The per-layer curves are notably high even at ckpt-800 (blue dashed) for Qwen3-VL models (GUI-Owl-1.5-8B and UI-Venus-1.5-8B), reflecting that these models have strong cross-platform generalization already encoded in their intermediate representations. The gap between ckpt-800 and last checkpoint is smaller on MMBench-GUI than on ScreenSpot-Pro, suggesting that the per-layer signal saturates earlier on this benchmark.

Per-Layer overlap@1 on UI-Vision
(A7 CrossAttn Probe, each probe layer evaluated independently)

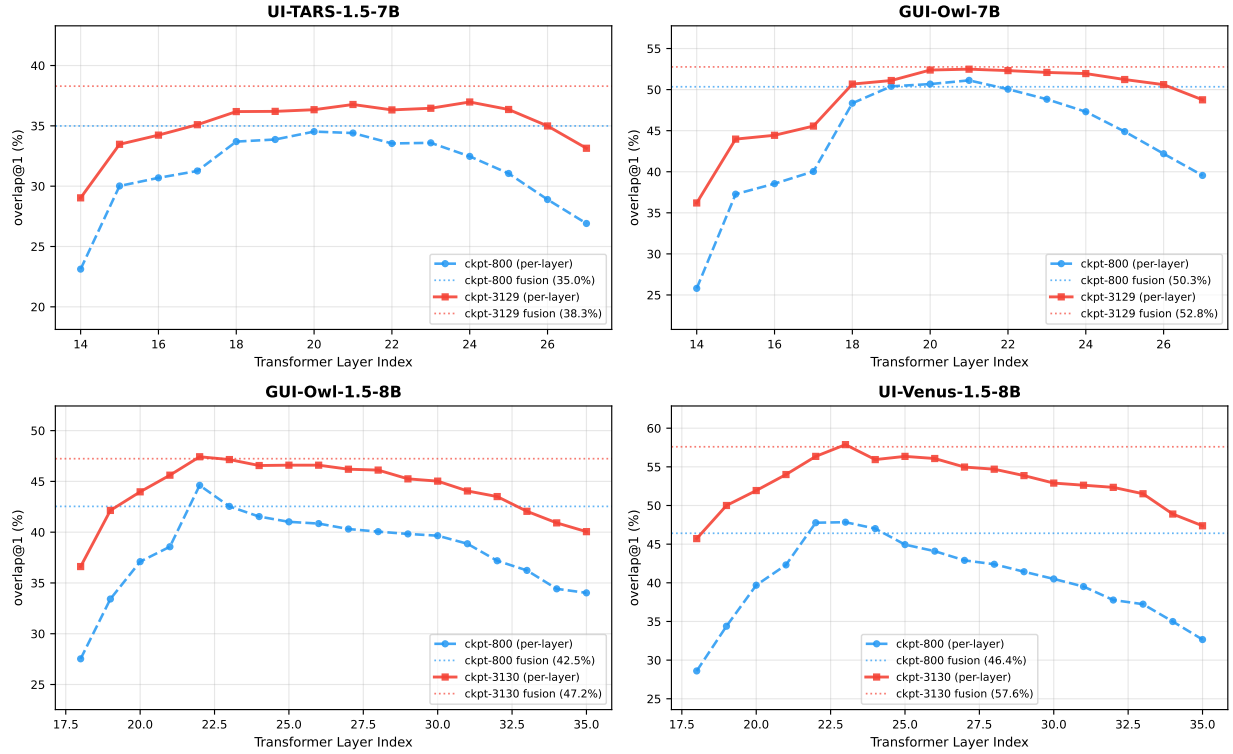


Figure 9: **Per-layer overlap@1 on UI-Vision.** UI-Vision decomposes grounding into Basic, Functional, Spatial, and Layout subtypes; accuracy here represents the weighted average over all 5,790 samples. Per-layer accuracy is substantially lower on UI-Vision than on other benchmarks, particularly for Qwen2.5-VL models at early training (ckpt-800 for UI-TARS-1.5-7B peaks below 30%), indicating that UI-Vision’s diverse task composition is harder for intermediate probes to solve with limited training. The inverted-U is still present but shallower and shifted earlier (peak around L18–L20), possibly because UI-Vision includes layout-level tasks that require higher-level semantic understanding.

Prompt for the UI-TARS-1.5 Model

```
1 You are a helpful assistant.  
2  
3 You are a GUI agent. You are given a task and your action history, with screenshots.  
4 You need to perform the next action to complete the task.
```

```
5  
6 ## Output Format  
7 Action: ...  
8  
9 ## Action Space  
10 click(start_box='<|box_start|>(x1,y1)<|box_end|>')
```

```
11  
12 ## User Instruction  
13 Task Instruction: {instruction}
```

Prompt for the GUI-Owl Model

```
1  You are a helpful assistant.
2  # Tools
3  You may call one or more functions to assist with the user query.
4  You are provided with function signatures within <tools></tools> XML tags:

5  <tools>
6  {
7      "type": "function",
8      "function": {
9          "name": "computer_use",
10         "description": "Use a mouse to interact with a computer screen."

11         "* The screen's resolution is ${height}x${width}."
12         "* Make sure to click any buttons, links, icons, etc with the cursor tip in the center of the
            element. Don't click boxes on their edges unless asked.",
13         "parameters": {
14             "properties": {
15                 "action": {

16                     "description": "The action to perform. The available actions are:"
17                     "* 'left_click': Click the left mouse button at a specified (x, y) pixel coordinate
                        on the screen.",
18                     "enum": ["left_click"],
19                     "type": "string"
20                 },

21                 "coordinate": {
22                     "description": "(x, y): The x (pixels from the left edge) and y (pixels from the top edge)
                        coordinates to click.",
23                     "type": "array"
24                 }
25             },

26             "required": ["action"],
27             "type": "object"
28         }
29     }
30 }
31 </tools>

32 For each function call, return a json object with function name and arguments within <tool_call></tool_call>
    XML tags:
33 <tool_call>
34 {"name": <function-name>, "arguments": <args-json-object>}
35 </tool_call>
36

37 ## User Instruction
38 Please complete the following tasks by clicking using 'left_click' function: {instruction}
```

Prompt for the GUI-Owl-1.5 Model

```
1  # Tools
2
3  You may call one or more functions to assist with the user query.
4
5  You are provided with function signatures within <tools></tools> XML tags:
6  <tools>
7  {"type": "function", "function": {"name": "computer_use", "description": "Use a mouse to interact with a
   computer.\n* The screen's resolution is 1000x1000.\n* Make sure to click any buttons, links, icons, etc
   with the cursor tip in the center of the element. Don't click boxes on their edges unless asked.", "
   parameters": {"properties": {"action": {"description": "The action to perform. The available actions are
   : \n* 'left_click': Click the left mouse button with coordinate (x, y) pixel coordinate on the screen.",
   "enum": ["left_click"], "type": "string"}, "coordinate": {"description": "(x, y): The x (pixels from the
   left edge) and y (pixels from the top edge) coordinates to click.", "type": "array"}}, "required": ["
   action"], "type": "object"}}}
8  </tools>
9
10 For each function call, return a json object with function name and arguments within <tool_call></tool_call>
    XML tags:
11 <tool_call>
12 {"name": <function-name>, "arguments": <args-json-object>}
13 </tool_call>
14
15 ## User Instruction
16 {instruction}
```

Prompt for the UI-Venus-1.5 Model

```
1  Output the center point of the position corresponding to the following instruction: {instruction}.
2
3  The output should just be the coordinates of a point, in the format [x,y].
```

Prompt for the UI-TARS-1.5 Retrofit Model

```
1  [System]
2  You are a GUI agent. You are given a task and a screenshot.
3  You need to perform the next action to complete the task.
4
5  ## Output Format
6
7  Action: ...
8
9  ## Action Space
10
11 click(start_box='<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>')
12
13 [User]
14 <image>
15 {instruction}
16
17 [Assistant]
18 click(start_box='<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>')
```

Prompt for the GUI-Owl-7B / GUI-Owl-1.5 Retrofit Model

```
1  [System]
2  # Tools
3
4  You may call one or more functions to assist with the user query.

5
6  You are provided with function signatures within <tools></tools> XML tags:
7  <tools>
8  {"type": "function", "function": {
9    "name": "computer_use",

10   "description": "Use a mouse and keyboard to interact with a computer, and take screenshots. This is an
        interface to a desktop GUI. You do not have access to a terminal or applications menu. You must click on
        desktop icons to start applications. * The screen's resolution is 1000x1000.\n* Make sure to click
        buttons, links, icons, etc with the cursor tip in the center of the element. Don't click boxes on their
        edges unless asked.",

11   "parameters": {"properties": {"action": {"description": "The action to perform. The available actions are:\n*
        'left_click': Click the left mouse button at coordinate (x, y) pixel coordinate on the screen.", "enum
        ": ["left_click"], "type": "string"}, "coordinate": {"description": "(x, y): The x (pixels from the left
        edge) and y (pixels from the top edge) coordinates to click. Required only by 'action=left_click'.", "
        type": "array"}}, "required": ["action", "coordinate"], "type": "object"}}}
12 </tools>

13
14 For each function call, return a json object with function name and arguments within <tool_call></tool_call>
    XML tags:
15 <tool_call>
16 {"name": "computer_use", "arguments": {"action": "left_click", "coordinate": [<|ground|><|pointer_start|><|
    pointer_pad|><|pointer_end|>]}}

17 </tool_call>
18
19 [User]
20 <image>
21 {instruction}
22

23 [Assistant]
24 <tool_call>
25 {"name": "computer_use", "arguments": {"action": "left_click", "coordinate": [<|ground|><|pointer_start|><|
    pointer_pad|><|pointer_end|>]}}
26 </tool_call>
```

Prompt for the UI-Venus-1.5 Retrofit Model

```
1 [User]
2 <image>
3 Output the center point of the position corresponding to the following instruction:
4 {instruction}.
```

```
5
6 The output should just be the coordinates of a point, in the format
7 [<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>].
8
9 [Assistant]
10 [<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>]
```



